

An evidence-led design story

Benchmark: 150 claims · 22 policies · 11 tables Outcomes: 4

Official result: Exp 32, 30/50, 0/37 false approvals

01 / PROBLEM

Why human attention, not expense processing, is the bottleneck

Imagine a large company processing thousands of expense claims a month. For a finance reviewer like Maya, the problem isn't that every claim is difficult — it's that every claim *arrives looking like it might be*. A hotel claim may be valid; a meal claim may be missing attendee details. Maya can't tell which until she checks the note, the policy, approvals, travel data and prior claims across several systems. The real bottleneck isn't expense processing — it's human attention.

ExpenseGuard is built to change that: resolve the routine cases automatically, request exactly the missing information when something is incomplete, and send only genuinely ambiguous or risky cases to a human — with the evidence already assembled.

Before and after, for one claim

Take a routine hotel claim.

Before:

Maya checks the note, the policy, the approval record and travel dates herself — a GBTA benchmark (context only) puts that at ~20 minutes, \$58 per report.

After, for claims the deterministic path resolves:

code checks the same four things in under a second, cited to the policy clause and approval record; only a failed check reaches Maya, evidence already assembled — one illustrative path, not how every claim is handled. 39 of 50 final-test claims (78%) never reached a human by some such path; residual cases needing a model add a median 1.3 seconds at ~\$0.00043 each. The change isn't fewer checks; it's the same checks, run before Maya opens the claim.

The question I wanted to answer wasn't "can I build an AI agent for expenses?" It was "what is the simplest architecture that can safely remove unnecessary human review without increasing the risk of a wrong approval?"

I tested deterministic rules, RAG, workflows and bounded agents across 55 experiments, each gating the next: can rules read enough? Is retrieval worth its cost? Do fixed tools beat agent tool choice? Do improvements survive fresh cases? The most important result was unexpected: the more capable guarded, tool-using design became more accurate — not because it had more autonomy, but because a code gate took discretion away from the model at the moment it mattered. I still didn't promote it, because it failed the safety bar on a true holdout.

02 / DESIGN CHOICES

What did RAG actually buy us, and who decides?

A no-policy model got 26/70 right (37.1%) — higher accuracy than tuned RAG managed. Full-context policy reached only 22/70 at \$0.271/70 claims; tuned RAG matched that score for \$0.053, five times cheaper, with traceable evidence. RAG's value was never a proven accuracy win; it earned its place as cheap, auditable grounding, confirmed by a policy oracle that moved accuracy only to 25/70 with the exact clauses handed over. Reasoning, not retrieval, was the bottleneck.

EXHIBIT A · DID RAG BEAT THE SIMPLER ALTERNATIVES?

VERIFIED

No policy		26/70 · 37.1%
Full corpus (\$0.271)		22/70 · 31.4%
Naive RAG		21/70 · 30.0%
Tuned RAG (\$0.053)		22/70 · 31.4%
Policy oracle (evaluator-on)		25/70 · 35.7%

N=70 each, gpt-4o-mini. Retrieval tuning raised recall@8 from 52.0% to 56.3% — coverage rose, the answer barely did.

Perfect evidence still left most claims wrong — policy application and enterprise checks, not clause-finding, were hard. A fixed workflow beat the model alone on dates, approvals and records (45/70) but falsely approved 7/52 non-approvable claims. The selective router traded two accuracy points (43/70) for 0/52 false approvals: code decides only when a rule fires and the facts sit in the note; the residual goes to an LLM. The guarded candidate pushes further with tools computing the disposition in code and a gate blocking overrides.

EXHIBIT B · HOW A CLAIM BECOMES A DECISION — BOTH ARCHITECTURES

EXPENSEGUARD / DECISION ARCHITECTURE

How a claim becomes a decision

Two tested paths · four possible outcomes · no payment authority · prototype, not deployed

SHARED INTAKE

Note + receipt

Decision-critical facts may be in free text; policy version and approvals matter.

Employee + claim facts

Policies + records

1 ROUTE

2 CHOOSE PATH

3 CHECK EVIDENCE

4 DECIDE / REVIEW

OFFICIAL FROZEN PROTOTYPE · EXP 30 → EXP 32

CODE FIRST

Deterministic resolver

rules_text + rules_v2
Read note facts; apply rules

DECISION POINT

Conclusive?

YES → use code decision
NO → residual path

RESIDUAL ONLY

Evidence + LLM

M4 RAG + resolved facts
LLM applies policy

CONCLUSIVE → DIRECT CODE OUTCOME

FOUR-WAY DECISION

Final disposition

Code or LLM outcome
APPROVE · REJECT · RI
ESCALATE → human review

GUARDED CANDIDATE · EXP 59 + EXP 60 GATE → EXP 61

CODE FIRST

Fixed routing

Simple cases close in code
Residual continues to agent

AGENT STEP

Choose policy + tools

Category compliance checks
Tool arguments from claim

TRUST BOUNDARY

Enforced code gates

Use tool disposition
APPROVE needs category tool

FOUR-WAY DECISION

Final disposition

APPROVE · REJECT · RI
ESCALATE → human review
Tool-content errors still possible

EVIDENCE THAT DETERMINED THE STATUS

Official · Exp 32 final test

30/50 correct · 0/37 false approvals
0/13 APPROVE recall

Candidate · Exp 60 holdout

34/50 correct · 0/30 false approvals
12/20 APPROVE recall · revised on this set

Candidate · Exp 61 preregistered

20/30 correct · 1/15 false approvals
9/15 APPROVE recall · not rerun

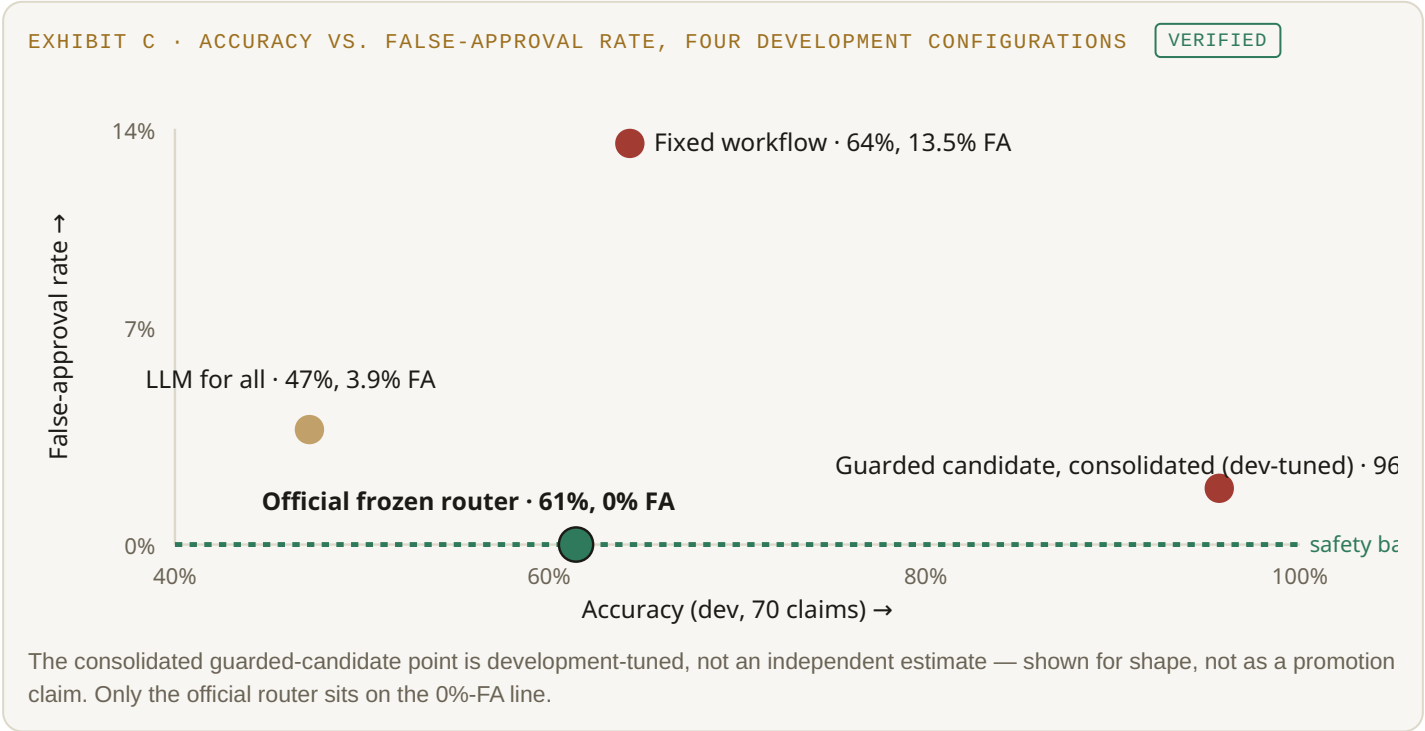
Why the candidate remains a candidate: Exp 61 approved a prohibited prepaid e-voucher even after its gift tool was consulted.

Source: repo Exp 30, 32, 59, 60, 61 records. Accuracy and false approvals use different held-out populations; no deployment effect is claimed.

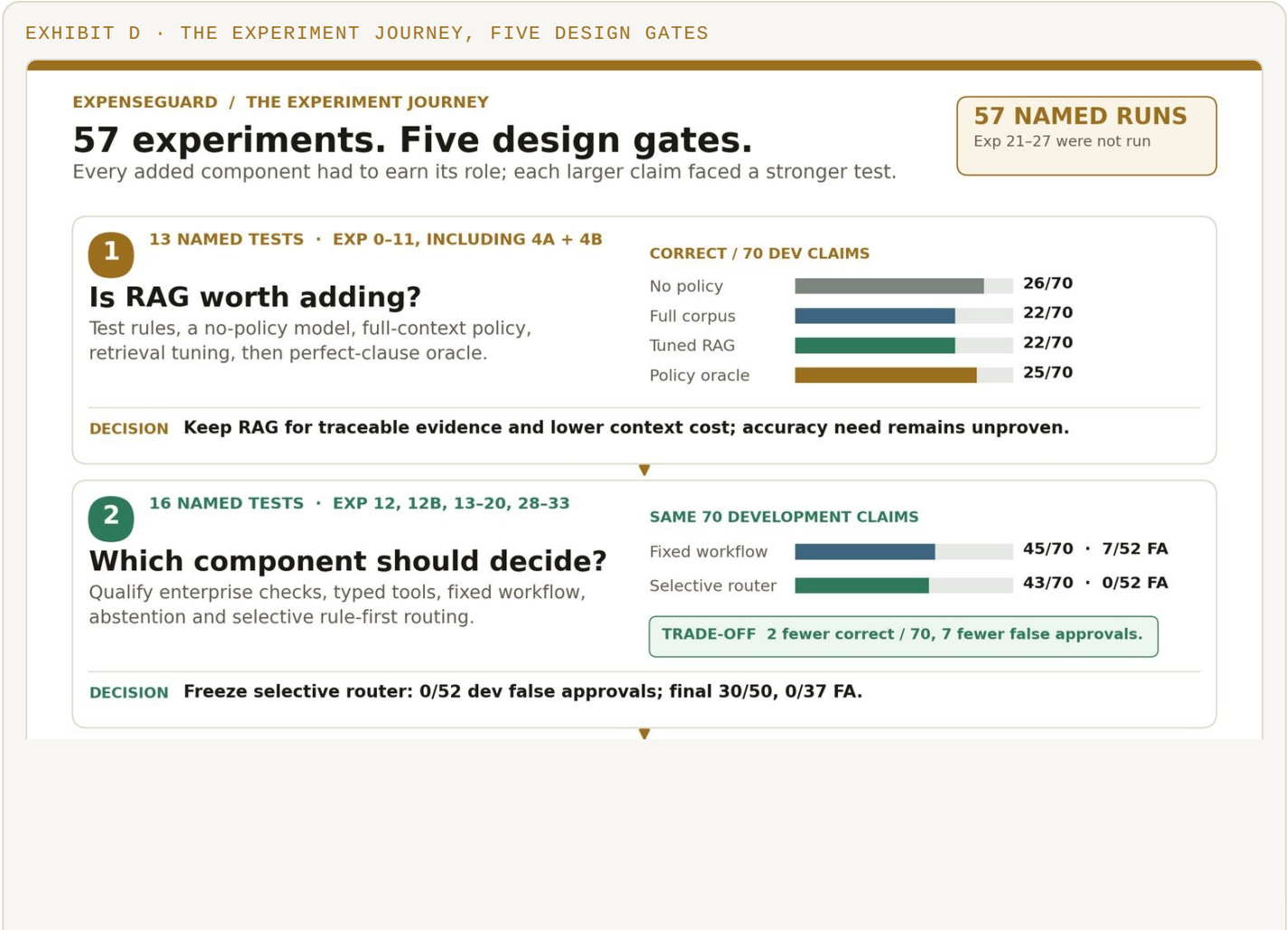
Two tested paths, four possible outcomes. The official path resolves conclusively in code wherever it can; the candidate routes through an agent step but is held to the same code-enforced trust boundary before anything is approved.

Accuracy, safety, and what survived fresh data

On the one-shot final test, the frozen design scored 30/50, 0/37 false approvals: deterministic cases 22/22, LLM-residual 8/28, none of 13 approvable claims approved.



Getting the guarded design this far took five gated rounds — agentic retrieval alone made things worse, a disposition gate fixed a narrow slice, and scaling it up surfaced new false approvals before settling (Exhibit D traces the arc).



3

11 NAMED TESTS · EXP 34-44

DIFFERENT DYNAMIC-CLAIM SLICES

Can an agent fix the residual?

Agentic RAG failed; better prompts and retrieval alone did not fix decisions. Move policy disposition into code.

4/13

Exp 34
agentic RAG

7/13

Exp 40
decision in code

17/19

Exp 44
guarded narrow slice

DECISION Guarded gate wins on a narrow slice; do not generalize 17/19 beyond it.

4

15 NAMED TESTS · EXP 45-59

CORRECT / 70 DEV · FA / 52

Does the guard survive full scale?

Extend category tools, fix live bugs and enforce code dispositions; test each change on 70 dev claims.

Exp 45



51/70 · 6 FA

Exp 52



44/70 · 0 FA

Exp 59



67/70 · 1 FA

DECISION Accuracy improved, safety fluctuated; the 67/70 result is development-tuned.

5

2 NAMED TESTS · EXP 60-61

FROZEN → GUARDED / SAME SET

Will the design survive new cases?

Compare frozen vs guarded on a fresh 50, then a preregistered independent 30-case stress set.

EXP 60 22/50 → 34/50 correct

Candidate 0/30 FA; label + tool gate revised on this set

EXP 61 11/30 → 20/30 correct

Candidate 1/15 FA; preregistered, not rerun

DECISION Keep guarded design as candidate: Exp 61 found a real gift-form false approval.

COST LENS · MEASURED INFERENCE VS MODELED OPERATIONS

Measured model spend / Exp 4B vs tuned RAG

\$0.271 vs \$0.053 per 70 claims

≈5.1× lower cost; same 22/70 accuracy.

Illustrative business model / different splits

\$19,170 vs \$12,916 per 1,000

Frozen Exp 32 final vs older Exp 52 dev. Not head-to-head.

MITIGATED IN CODE

Disposition gate · category-tool consultation

Conflict handling · bounded agent/tool calls

These checks reduce specific failure modes.

STILL OPEN / NOT PROVEN

Gift-form free-text parsing · retrieval injection

Human-note transfer · finite-sample safety

No system is deployed or authorized to pay claims.

Source: docs/README.md; Exp 4B, 30, 32, 34, 40, 44, 45, 52, 59, 60, 61; cost_and_business_impact.md.

Counts summarize named experiment entries (13 + 16 + 11 + 15 + 2 = 57). FA = false approvals among ground-truth non-APPROVE cases.

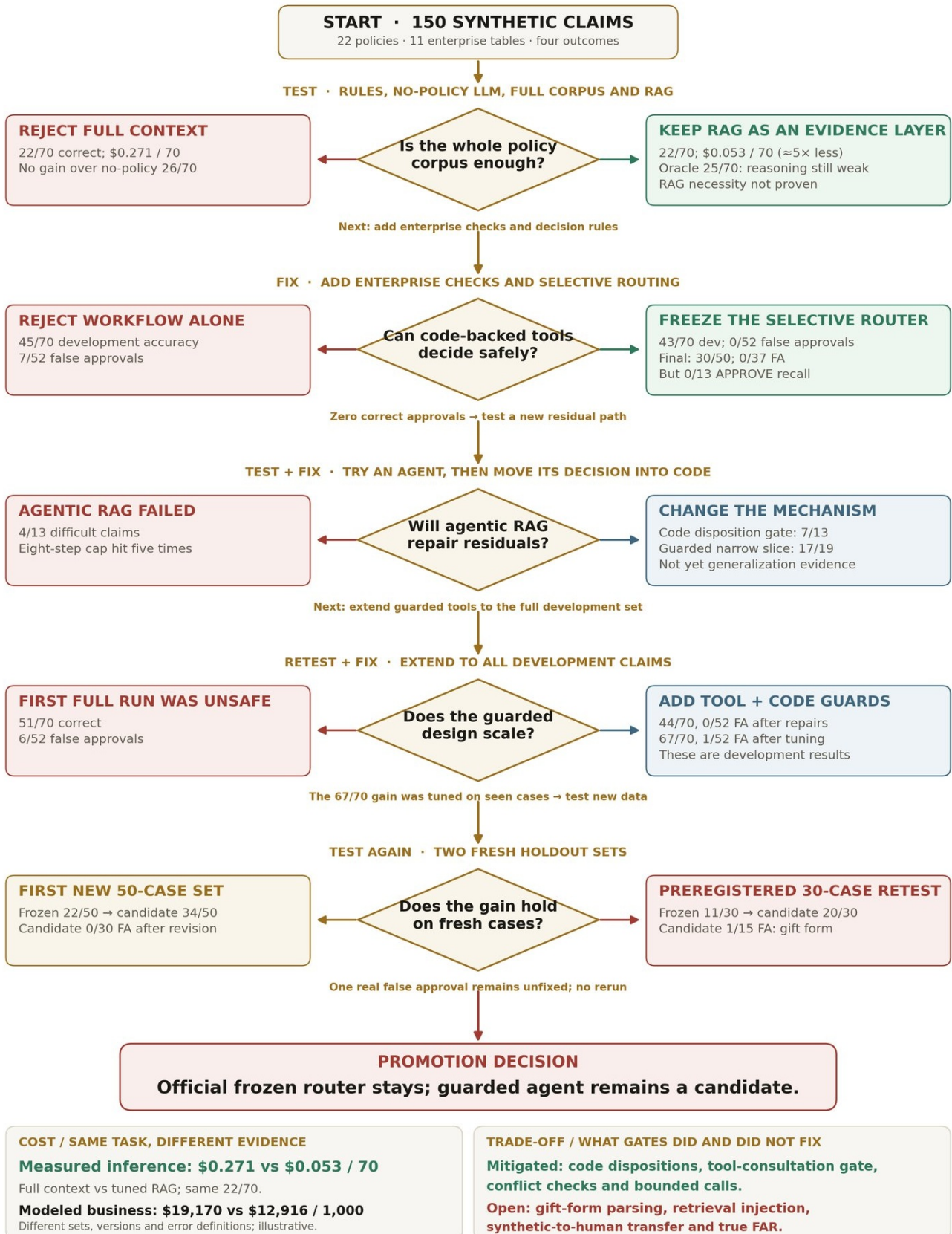
Each gate's decision fed directly into the next round's design, not the other way around — nothing here was decided before it was tested.

Fresh data was the real test. On a new 50-case holdout (Exp 60), the frozen resolver scored 22/50 and the candidate 34/50 — not a clean, untouched result, as Section 05 explains. The preregistered 30-case stress set (Exp 61) is the stronger evidence: cases generated only after the design was locked, favoring the candidate on accuracy (20/30 vs. 11/30) but catching a real false approval (1/15) — a prepaid e-voucher its own gift-compliance tool still missed.

EXPENSEGUARD / EXPERIMENT-TO-DECISION FLOW

Test. Fix. Retest. Challenge.

57 named experiments grouped by what each phase changed · prototype, not deployed



How to read the flow: side branches show the observed result at each gate; the central arrows show the next experiment decision.

FA = false approval among non-APPROVE cases. First holdout was revised on its own set; second holdout was preregistered.

Sources: docs/README.md; docs/exp04b, exp11, exp30, exp32, exp34, exp40, exp44, exp45, exp52, exp59, exp60, exp61; cost_and_business_impact.md.

The promotion decision at the bottom is the one that matters: the frozen router stays official, the guarded agent remains a candidate, because the one false approval found at the end was never fixed and rerun.

QUESTION	PIVOTAL TEST	WHAT CHANGED	DECISION
Is RAG needed?	No policy, full corpus, tuned RAG, oracle	Retrieval improved grounding, not accuracy	Keep RAG as evidence layer
Who should decide?	Fixed workflow vs. selective router	Fewer false approvals with code-first routing	Freeze selective router
Can an agent fix residual cases?	Agentic RAG, then code disposition gate	Tool choice alone failed; code gate helped	Build guarded candidate
Does it scale beyond a narrow slice?	Full development set and fixes	Accuracy rose, safety fluctuated	Test on fresh cases
Does it generalize?	Exp 60 and preregistered Exp 61	A real false approval appeared	Do not promote candidate

04 / BUSINESS & EVALUATION CRITIQUE

Why accuracy alone is the wrong headline

The official design's 0/37 false approvals hid 0/13 APPROVE recall: of the 20 Exp 60 claims that should have been approved, the frozen router approved none.

EXHIBIT F · WHAT THE DESIGN CHANGES FOR FINANCE – OPERATING QUEUE, EMPLOYEE IMPACT, AND COST SENSITIVITY

VERIFIED

EXPENSEGUARD / BUSINESS DECISION VIEW

What does the design change for finance?

Same fresh claims for the service comparison. Costs below are a separate, illustrative model.



02 EMPLOYEE LENS · 20 CLAIMS THAT SHOULD HAVE BEEN APPROVED

Frozen router



Guarded



Why this matters

The frozen design protected against wrong approvals but blocked every legitimate approval. The candidate recovered 12, while six still needed a reviewer and two were wrongly blocked.

03 SAFETY BAR · A SECOND HOLDOUT CHANGED THE PROMOTION DECISION

FIRST FRESH SET / 50 CASES

Candidate: 0/30 false approvals

Result revised after a label correction and tool gate.

PREREGISTERED RETEST / 30 CASES

Candidate: 1/15 false approvals

Gift-form tool missed a prohibited prepaid e-voucher.

Decision: keep the frozen router official; the guarded design needs a fix and another preregistered test.

04 COST LENS · ILLUSTRATIVE BASE SCENARIO, NOT A HEAD-TO-HEAD TEST

Modeled dollars per 1,000 claims

■ Human review ■ Wrong REJECT as coded ■ Unneeded RI

Frozen / original final test



\$19,170

Review \$5,170 · wrong REJECT \$13,000 · unneeded RI \$1,000 · model < \$2

Earlier guarded / development



\$12,916

Review \$8,057 · wrong REJECT \$3,571 · unneeded RI \$1,286 · model < \$2

The candidate row is an older version and a different sample. The model also uses a different wrong-decision definition from the service chart. Directional sensitivity only; no savings claim.

BUSINESS CONCLUSION

The opportunity: fewer wrongly blocked claims. The limit: safe approval on unseen data.

Source: saved Exp 60 and Exp 61 comparison_result.json; docs/cost_and_business_impact.md. Prototype, not deployed; no real-world savings measured.

Costs are an illustrative base scenario, not a head-to-head test: the candidate row uses an earlier version and a different sample. No savings are claimed.

An early cost model omitted false-rejection and unnecessary-info costs. Correcting it moved the comparison — Exhibit F shows the two live scenarios, \$19,170 vs. \$12,916, different samples and candidate versions, not a declared winner. A third, diagnostic scenario (\$20,393, reusing original final-test cases without authorization) sits in `docs/cost_and_business_impact.md`, not the exhibit. The model's \$50 false-rejection price also fires only on a predicted REJECT; an approvable claim sent to REQUEST_INFORMATION is priced at \$10 instead, narrower than Exhibit F's own definition.

Exp 60 also shows why accuracy alone is insufficient: the candidate resolved 33/50 correctly without review, versus 21/50 for the frozen router, but escalated 13 claims against one. Its modeled cost advantage uses an earlier version and a different sample — a scenario, not measured savings. The design must earn its extra review workload while still meeting the false-approval bar.

Business feasibility, plainly: direct model/API cost is under \$0.50 per 1,000 claims — inference only. The frozen router's modeled *total* operating cost, review and error costs included, is \$19,170 per 1,000. It has held its safety bar every time, but 0/13 APPROVE recall makes it a compliance gate today, not a labor-saving tool. A fresh independent holdout is the next promotion gate for the candidate; production feasibility for either design would still need human-authored data, the Section 06 shadow-mode evaluation, and security hardening.

Critical evaluation: how each fix exposed a deeper failure

I began by treating policy retrieval as the likely bottleneck. It was not, as Section 02's oracle result shows — the model had the evidence but applied it poorly. Moving dates, thresholds and enterprise checks into code helped, yet the fixed workflow (Section 02) falsely approved non-approvable claims. I froze a selective router that gave code authority where its answer was conclusive, leaving uncertain cases to the residual path.

The final test exposed the cost of that safety choice: no false approvals in 37 opportunities, but none of the 13 claims that deserved approval, approved. A guarded agent recovered legitimate approvals by computing dispositions in tools and blocking model overrides — solving one reasoning failure while creating a new dependence on whether the tools understood the claim correctly. Exp 61 found that gap exactly: the gift tool was called, but missed a prohibited prepaid e-voucher, producing 1/15 false approvals.

The evidence itself has limits. Exp 60 was revised after a label correction and a gate added on the same set; Exp 61 is the cleaner challenge. Cases are synthetic, mostly drafted by the inference model's own family. The Exp 32 manifest no longer verifies today's regenerated files — caused by later dataset regeneration, not a runner flaw; separately, the exploratory shared runner should save predictions before loading labels, as Exp 32's own dedicated runner already did. With 0/37 false approvals, the Wilson 95% upper bound is still 9.4%; security was assessed against OWASP 2026, prompt injection still open. The next test should freeze labels first, use independently written notes, and check tool interpretations before promotion.

A testable path to promotion

Two things next. A fresh, independently authored holdout, frozen before generation, every label reviewed first, with generalized tool contracts in place of the category-specific patches that caused most of this project's real bugs. Then a human shadow-mode evaluation — the frozen router running alongside real reviewers, decisions compared but not acted on.

The frozen selective resolver remains the official research architecture not because it is the most capable design I built, but because it is the only one with a predeclared frozen evaluation and a result that cleared the safety bar. The gap between best-performing and most trusted is the central finding of this project.

REASONING DEPTH RAG ablations, oracle tests, the code-gate repair saga, two independent fresh holdouts.	OUTCOME CRITIQUE False approvals vs. false rejections vs. APPROVE recall; data and runner-ordering limits disclosed.
BUSINESS CASE Decision-authority trade-offs, the operating-trade-off exhibit, cost-sensitivity scenarios.	STRUCTURE & PATH A question-to-decision narrative with a preregistered next test, not a feature list.

Full experiment record (55 experiments): docs/README.md and individual docs/exp*.md files. Core sources: exp04b_long_context_baseline.md, exp09_metadata.md, exp11_oracle.md, exp30_selective_router.md, exp32_final_test.md, exp33_failure_analysis.md, exp40_decision_in_code.md, exp52_validation_tuned_candidate.md, exp59_final_fix.md, exp60_fresh_holdout.md, exp61_v3_holdout.md, cost_and_business_impact.md, owasp_llm_top10_2026.md. Figures were re-derived from available saved result artifacts where possible; known reproducibility exceptions, including Exp 32's regenerated-dataset manifest, are disclosed in Section 05.